



PRD - Draft

▶ 목차

ProtoPie Connect Cloud PRD

작성일: 2026-04-26
마지막 업데이트: 2026-05-16
상태: 초안
대상 독자: PO / PM / 디자이너 / 엔지니어 모두

0. Executive Summary — 한 페이지

본 §0은 5분 reader용 lean summary. 같은 정보의 다른 깊이 cut이며 §0가 충돌하면 본문이 source of truth.

확정: 신규 Connect는 앱을 새로 작성한다 (레거시 코드 마이그레이션 아님). 기존 Connect의 모든 기능을 가져감 (UX:stageview는 일부 개선 포함되나 핵심 가치는 아님). 로컬 서버 모드와 클라우드 통신을 동시 지원하는 hybrid. Beta는 Connect 기능이 클라우드 환경에서 작동하는지 검증한다 (분산 하드웨어 협업은 클라우드화로 가능해지는 시나리오 중 하나일 뿐, 클라우드화의 목적은 아님).

타겟: B2C (P1 개인·P2 소규모 팀) + B2B (P5 회사·조직 멤버 — 현재 Connect 사용자 base의 다수) 하이브리드.

🎯 Beta 최우선 2가지 — 모든 결정의 기준

이 두 항목 위에 다른 우선순위는 없다. 모든 trade-off·범위 조정·deferral 결정 시 본 순위가 tie-breaker.

- 기존 Connect → 신규 앱 전환 — 레거시 사용자 가시 기능을 신규 앱에서 동등 이상으로 보장. 코드는 재작성, 기능은 보존 (§1-4·§1-5 정합)
- 클라우드 환경에서 Connect 기능 사용 가능 — 모든 핵심 워크플로가 cloud-mediated 환경에서 작동 (분산 하드웨어 협업·stageview·Stage 협업 등 §6-6·§6-7)

이외 항목(Debugger 모드 F-BRG-16, HTML import F-BRG-17 등 신규 기능, 운영 메트릭, UX 개선)은 모두 위 2가지의 하위 또는 보조 위치. 위 2가지가 위협받으면 다른 항목은 deferral 대상.

일정: 개발 5/11~6/15 (5주, user research·기획·디자인·엔지니어링 모두 포함) + QA 6/16~6/30 (2주) → 출시 2026-06-30 (§2-0)

Goals

Beta 종료 시점에 어떤 결과가 나오면 성공인가? — §0 최우선 2가지 priority의 직접 측정.

Priority 검증 (필수, Beta의 전부):

- 🔄 Priority #1 — 레거시 → 신규 앱 완전 재작성: 레거시 사용자 가시 기능 47개가 신규 앱에서 동등 이상으로 작동. 코드는 새로 작성, 기능은 보존
- ☁ Priority #2 — 클라우드 환경에서 Connect 기능 구현: 모든 핵심 워크플로가 cloud-mediated 환경에서 작동·반복 사용. cloud-only 환경에선 Connect가 정상 운영 가능한 수준에 도달

보조 outcome (공통 토대) — 위 2개를 떠받치는 운영 수준:

- 👤 오보딩 정착: 가입자가 Beta 종료 시점에 Pie.Stage 를 자력으로 만들고 본인 워크플로에 정착
- 🛡 격리·운영 신뢰성: cross-Team 유출 사고 0건, Multi-AZ failover 정상 동작

시나리오별 검증 (S10-S11-S12 등)은 Goals가 아니라 §5에서 다루는 측정 항목. S10 분산 하드웨어 협업·S11 stageview 개선·S12 Stage 모델은 모두 Priority #1-#2가 작동하는지 증명하는 신호이지, Beta의 목적 자체는 아님. 클라우드화의 목적은 “Connect 기능이 클라우드에서 작동”이지 특정 시나리오를 위한 것이 아니다.

수치 임계값(Go/No-Go·시나리오별 성공 기준 등)은 미결정. 레거시 Connect 사용 데이터 추출 후 baseline을 도출하고, 그 baseline 대비 비교 기준을 §2-3에 추가한다 — 본 §0 Goals는 정성 outcome만.

상세: §2-3.

Users

누가 이 도구를 쓰며, 무엇을 하려 하는가?

페르소나	역할	핵심 needs
P1	개인 사용자 (B2C)	본인 PC에서 빠르게 프로토타입 시연. 레거시 동등 사용성 + 클라우드 동기화
P2	소규모 B2C 팀	팀원 초대로 Pie·플러그인·Stage 공유. 결제·멤버십을 1 Team에 모음
P5	B2B 팀 멤버 (현재 Connect 사용자 base 다수)	회사·조직 단위 결제·멤버 간 자료 공유. 중앙 거버넌스 빠른 파이 공유와 협업이 회사 보안·정책 안에서 가능
P4	하드웨어 통합 개발자	각자 PC에 가진 하드웨어를 클라우드 세션에 통합 (S10 시나리오 검증자)

상세: §4.

Problem

고객은 지금 무엇에 막혀있는가? 우리가 풀면 충성도·사용률은 어떻게 올라가는가?

현재 Connect 사용자(다수가 B2B 회사 고객)의 실제 피드백 기반 누적 friction (구체 고객 evidence는 §1-2-1 요약·📌 STRATEGY - WIP 부록 A 상세 참고):

- LAN-only / localhost bound — Stageview.웹 임베드를 모바일/원격에서 사용 불가. Toyota Japan, TomTom, Amazon, Zoox+Aston Martin, Canny 200+ vote(가장 오래되고 지지 많은 요청)
- 외부 공유·원격 협업 차단 — API 연동 prototype을 외부 공유 못 함, ngrok 우회는 회사 IT가 차단. Zillow Group, Disney Streaming
- 하드웨어 결합 작업이 단일 PC에 갇힘 — UT.prototype 테스트가 회사 멤버 PC에 분산된 하드웨어를 통합 못 함. Mindray Medical, Inovance Technology, Midea (intelligent hardware/medical/home appliance)
- 단일 사용자 가정 → 회사 거버넌스 부재 — 보안·로컬 배포 요구 미충족. Huawei, BMW China, BYD, GM (high-security 기업)
- 차량 플랫폼 성능·안정성 한계 — 고충실도 prototype 실행 시 명확한 lag, 대용량 Pie 안정성 문제. Seres, Great Wall Motor(장성자동차), Voyah, Lingshu, Pan Asia, American Honda
- 메모리 인증·매일 반복 friction — 앱 재시작마다 로그인·PIN 손실 (cumulative daily friction)

외부 정량 evidence: 2026년 2개월간 중국 25개 핵심 기업 방문 (전부 B2B) — 자동차 OEM 13곳·인텔리전트 하드웨어·의료기기·인터넷·가전 등 100-billion groups-unicorns-head enterprises. 4가지 핵심 이슈(performance bottleneck · data integration · product functions · deployment)이 위 friction과 일치 — 📌 STRATEGY - WIP 부록 A-11 참고.

이 friction이 사용 빈도·만족도·충성도를 깎고 있음. cloud 전환의 주된 목표는 시장 트렌드 추종이 아니라 고객 니즈 충족 → 충성도·사용률 제고:

- §1-3 핵심 가치 #1 (순쉬운 접근) — 누적 friction 일괄 해소 → 매일 사용 만족도·재방문률 상승
- §1-3 핵심 가치 #2 (빠른 파이 공유와 협업) — 회사 워크플로우에 정착 → 이탈 감소·도입 확산

시장의 SaaS·실시간 협업 흐름(Figma·Framer 등)은 사용자 기대치를 높이는 보조 컨텍스트. 우리 사용자도 같은 기대를 가지지만, 결정 동기는 우리 사용자의 누적 friction 자체.

상세: §1-2 (레거시 한계 11개) · §1-2-1 (고객 피드백 요약) · §1-3 (사용자 관점 핵심 가치 2가지).

Scope

Beta에 무엇이 들어가고 무엇이 빠지는가?

들어감 — 기능 기준 2x2 분류:

기존 기능 유지·개선 (레거시 동등 보장)

- 사용자 노출: USB/Serial/MQTT 하드웨어 통합 / 커스텀 플러그인 / PIN 인증 / IFTTT-API-폰트 / 로컬 서버 모드 / Studio 연동 (STU-1-STU-2)
- 백엔드: 인증·세션 영속화 (메모리 → DB:Redis) / 빌드 보호 강화

신규 기능 개발

- 사용자 노출: 분산 하드웨어 협업 (S10) / Team 내 Stage 모델 (디스코드 채널) / 클라우드 통신 모드 (hybrid의 새 축) / Web Dashboard / Debugger 모드 (F-BRG-16) / Socket 기반 HTML 프로토타입 import (Beta 실험적, F-BRG-17)
- 백엔드: @ppc/local-server (Fastify + Socket.IO) 신규 작성 / 단일 모노레포 (apps/web + apps/desktop + apps/local-server) / 2-process Electron (main + child fork) / Multi-tenancy 아키텍처 — v0.7.0: Team 이 root entity, Device 도 Team-scoped, user identity 는 ProtoPie Cloud SoT / 격리 메커니즘 (자동 필터-RLS) / 수평 확장 stateless 서버

안 들어감: AI 기능 (Bridge IDE의 Claude SDK 등), 플러그인 마켓플레이스·서명, 3rd-party 플러그인 생태계, on-prem, Team 간 자원 공유 등

상세: §2-0 (들어감) · §3 (안 들어감).

Key Flows

§1-3 핵심 가치 2가지가 사용자 워크플로우에서 어떻게 작동하는가? — 검증 시나리오 3건. 단 시나리오는 priority 작동을 보여주는 신호이지 Beta의 목적 자체는 아님 (§0 priority frame 참고).

- S10 — 분산 하드웨어 협업 (핵심 가치 #2 빠른 파이 공유와 협업 검증): 멤버 A가 Arduino, 멤버 B가 MIDI를 각자 PC에 연결 → 같은 Stage 에서 Bridge에 Pie 로드 → 한 Pie 세션이 두 PC의 하드웨어를 동시 사용. 레거시: 시연 시 모든 하드웨어를 한 PC에 모아야 함 → Beta: 각자 PC에서 협업
- S11 — stageview 미러링 (핵심 가치 #1 손쉬운 접근 검증, 단순 개선): Bridge가 stageview 시작 → 모바일/웹 viewer가 Cloud Relay 경유 미러링 + 시청자가 Pie 목록에서 시청할 Pie 선택. 레거시: LAN bound·모든 Pie 강제 시청 → Beta: 어디서나 선택적 Pie
- S12 — Stage 생성·초대·격리 (핵심 가치 #2 빠른 파이 공유와 협업 검증): 사용자가 Team 안에 Shared Stage 생성 → 멤버 초대 (role: editor/viewer) → Pie/플러그인/Relay 세션이 Stage 단위로 격리 (디스코드 채널 모델). 레거시: 자원 분리 부재 → Beta: 회사·팀 워크플로우에 정착 가능

상세: §5 (S1 / S1-2 / S3 / S8 / S10 / S11 / S12 + S-ERR 카탈로그).

Open Questions

아직 결정 안 끝난 항목은 무엇인가?

- D2 + D12 메시지 envelope + 하드웨어 효과 idempotency 정책 (per-Bridge monotonic seq + Redis dedupe window) — ARCH §8-1
- D4 Bridge 빌드 보호 구제 수단 조합 (asar 암호화 / JS 난독화 / 네이티브 모듈) — ARCH §8-1
- §2-3 정량 목표 임계값 — 레거시 Connect 데이터 추출 후 baseline 비교 기준으로 추가

상세: ARCH §8-1 (D-항목) · ARCH §9 Week 1 (Launch checklist).

1. 배경

1-1. 현재 제품: ProtoPie Connect (Legacy)

`protopie-connect/` 디렉토리는 현재 운영 중인 ProtoPie Connect 제품의 코드베이스.

이미 상용 운영 중인 제품이며, 신규 Beta는 이 제품의 차세대 버전을 만드는 작업이다.

현재 판매 모델

- ProtoPie Cloud의 **Addon** 형태로 판매. 사용자가 Cloud 계정으로 **Addon**을 구매해야만 Connect 사용 가능

즉 **Connect**는 무료 도구가 아니며, 신규 Beta도 동일한 유료 구조를 유지. 확장한다. 단, B2B Cloud 환경에 대해서는  **STRATEGY - WIP** 문서에서 다시 이야기 한다.

현재 가치 제안:

- ProtoPie 프로토타입을 실시간 멀티 디바이스 환경에서 테스트
- USB / Serial / MQTT를 통한 하드웨어 통합 (Arduino, IoT, 게임패드)
- ProtoPie Cloud에서 프로토타입 단방향 다운로드
- PIN 기반 로컬 네트워크 원격 접근
- ZIP 기반 커스텀 플러그인 실행 (IFTTT, API 등)

핵심 형태: 같은 Node.js 서버 코어를 두 가지 모드로 제공.

- **Desktop** 모드: Electron 셸로 감싼 데스크톱 앱 (macOS / Windows). 인증은 두 갈래 — (a) ProtoPie Cloud OAuth 로그인 (Cloud Relay·동기화 사용) / (b) 로그인 X · 라이선스 키 만 입력 (LAN 단독 운영, 로컬 DB 만 사용, Cloud 의존성 0 — 레거시 **LicenseManager** .lic 후신). schema 는  **DB_SCHEMA_LOCAL - Draft** 별도, Cloud DB 와 동기화 없음
- **Server** 모드 (**RUNMODE=server**): Linux 등에서 헤드리스 데몬으로 동작 — R&D 랩, 키오스크, 자동화 환경 (Beta scope-out: GUI 모드만)

1-2. 레거시의 구조적 한계

중요: 레거시 기능(stageview, 플러그인, 하드웨어 통합, 로컬 서버 등)은 모두 신규 앱에서 동등 이상으로 보장된다. 한계는 그 기능들이 올라타는 토대(아키텍처·운영·보안·확장성)에 있다.

각 한계 항목 끝의 고객 cite는 §1-2-1 요약-STRATEGY 부록 A 상세의 실제 고객 피드백 근거. 일부 항목은 architectural-internal 성격이라 cite 없음.

- 하드웨어 결합 작업의 공유·협업 차단: 시연·테스트가 작동하려면 모든 하드웨어를 한 PC에 강제로 모아야 함 → 회사 멤버끼리 하드웨어·작업·결과를 공유 못 함. 각자 PC에 흩어진 하드웨어로 협업하는 워크플로우 불가능 (§1-3 핵심 가치 #2 “빠른 파이 공유와 협업”의 가장 큰 차단 요인). Cite: *Mindray Medical·Inovance Technology·Midea (UT+Connect+hardware), BMW China (steering wheel·pedal·rotary 자동차 물리 입력)*
- **SaaS화 토대**: Server 모드가 있지만 단일 사용자·메모리 상태라 멀티테넌트 배포 부적합. Cite: *Canny 200+ vote — “ProtoPie Studio/Connect 웹 버전 + 실시간 협업”이 가장 오래되고 지지 많은 요청. 2026 China visit 25개 B2B 기업 모두 SaaS-grade 토대 기대*
- 사용자 모델 — 단일 사용자 가정: 한 사람만 쓰는 구조 → 회사·팀 단위 운영 불가. Cite: *모든 B2B 고객 (Huawei·BMW·BYD·GM·Mindray·Inovance·Midea 등 100% B2B in 2026 China visit) — 회사 단위 사용 전제*
- 인증 상태: 메모리 저장 — 앱 재시작 시 모든 로그인·PIN 손실 (매일 사용자에게 매일 반복되는 friction)
- 확장성: 단일 프로세스 — 사용자 늘어도 서버 추가 불가 (internal/architectural)
- 클라우드 연동: 단방향 다운로드만 — 공유·협업·동기화 토대 없음. Pie-작업 결과를 클라우드 통해 회사 멤버와 공유하는 흐름 부재. Cite: *Zillow Group (API 연동 prototype 클라우드 호스팅 외부 공유 필요, ngrok 우회 회사 IT 차단), Disney Streaming (외부 네트워크 원격 UT 필요), Toyota Japan/TomTom/Amazon (Stageview 모바일 웹 임베드 필요)*
- 플러그인 보안: 격리 없음 — 임의 프로그램이 사용자 PC 전체 접근 가능. Cite: *Huawei·BMW China (보안 강한 기입의 격리 deployment 요구)*
- 코드 구조: 모놀리식 — 부분 교체 어려움 (internal/architectural)
- 모니터링: 로깅만 — 운영 지표·장애 추적 도구 없음 (internal/operational)
- DB — 사용자 PC 안의 파일: 클라우드 백업·이전 어려움 → 입력 변동 시 자산 소실. Cite: *NIO — 1년 사이 팀 변동으로 proficient user 거의 없음, 과거 training 성과·asset 모두 소멸 (2026 visit report)*
- 멀티테넌트: 개념 자체 없음 — 회사·조직·팀 구분 못 함 → 중앙 결제·거버넌스·라이선스 통합 불가. Cite: *모든 B2B 고객의 회사 단위 운영 needs (P5 페르소나 §4-4 직접 evidence)*

신규 Connect의 작업 = “기능 추가”가 아니라 “동일 기능을 SaaS·멀티테넌트·클라우드 협업 가능한 토대 위에 재구현” (UX-stageview는 부수적으로 일부 개선 — Beta 핵심 가치 아님). 항목별 신규 해소는 §1-4 비교 매트릭스 참고.

1-2-1. 사용자 요구사항 — 실제 고객 피드백 evidence

다음은 레거시 Connect 사용자(다수가 B2B 회사 고객)의 실제 요구사항·피드백 요약. §0 Problem-§1-3 비즈니스 동기-§4-4 P5 페르소나-F-BRG-16 Debugger 모드 등 본 PRD 전반의 구체적인 근거 데이터. 전체 고객별 상세는  STRATEGY - WIP 부록 A 참고.

Source: ① 글로벌 고객 individual 요구사항·Canny 투표 (A-1~A-10) + ② 2026 Customer Visit Report — 중국 25개 B2B 기업 2개월 심층 방문 (A-11). 두 소스 모두 동일한 4가지 핵심 이슈(performance-data integration-deployment-product functions)에서 강한 cross-validation.

#	카테고리	핵심 요구 (요약)	대표 고객	본 PRD 매핑
1	웹/클라우드 기반 Connect	API 연동·외부 공유·Stageview 모바일·웹 임베드	Zillow / Disney / Toyota / TomTom / Amazon / Zoox+Aston Martin / Canny 200+ vote (가장 오래·지지 많은 요청)	§0 priority #2 직접 evidence — 클라우드 환경 작동
2	Connect 환경에서 UT	Connect + UT 통합 (아이트래킹·히트 맵·녹화 등)	J&J / Mindray / Inovance / Midea / 현대차 / Google Wearable / Samsung DA	UT는 Beta scope-out, Post-Beta candidate (수요 증명 evidence)
3	오프라인 / 온프레미스	보안 환경 오프라인 모드	BMW China / 현대차 / Huawei / BYD / GM	Beta scope-out (On-prem Post-Beta). 단 §6-4 hybrid 로컬 모드는 부분 충족
4	성능·안정성	차량 플랫폼·대용량 Pie 안정성	Seres / 장성자동차 / Voyah / Lingshu / Pan Asia / American Honda	§7 비기능 운영성 evidence
5	Stageview 제어	카메라/Unity 레이어 동적 제어·온보딩	Continental / Scania / 내부	§6-7 stageview (단순 개선 범위)
6	API	Keep-alive-data binding-AI Connect	Zoox / GM / J&J / 42dot	§6 plugin-integration evidence
7	Multi 디바이스 / 스크린	여러 Pie/디바이스 동시 제어	GM / Scania (최대 5 Pie) / Zoox+Aston Martin	S10-S11-§6-6 Relay 시나리오 evidence
8	파일·설정 관리	설정 백업·LocalSend 탐지	Rivian / CT / 내부	운영 §6-1-§6-3 evidence
9	디버깅 UX	message flow tracing (source→destination)	Scania / 내부	F-BRG-16 Debugger 모드 직접 evidence
10	하드웨어 연동	자동차 프로토콜 / RPi5	BMW China / Scania / ProtoPioneers	§6-4 Bridge·하드웨어 통합 evidence

이 표가 보여주는 것 — 가장 큰 요구는 카테고리 #1 “웹/클라우드 기반 Connect” (Canny 200+ vote = 가장 오래되고 지지 많은 요청). 이는 §0 priority #2 (클라우드 환경에서 Connect 기능 구현)와 정확히 일치 — Beta scope의 직접적 근거. 또한 고객 다수가 자동차 OEM·헬스케어·가전 등 B2B 회사라는 점이 §4-4 P5 페르소나(B2B 다수)의 직접적 근거.

요구 ↔ Beta scope 정합성 결론:

- Beta 직접 충족: #1 (P#2 직결), #5·#6·#7·#9·#10 (관련 시나리오·기능에 부분~전부 포함)
- Post-Beta deferral: #2 UT, #3 온프레미스 — 본 PRD §3 Non-goals와 정합 (수요는 분명, Post-Beta 우선 대상)
- Beta 운영 측면: #4 성능·안정성, #8 파일·설정 관리 — §7 비기능·§6 운영 요구사항으로 흡수

1-3. 비즈니스 동기 — Why now?

본 절의 결정 동기는 고객 니즈 충족 → 충성도·사용률 제고 — 시장 트렌드 추종이 아니다. 시장 흐름은 사용자 기대치 상향의 보조 컨텍스트일 뿐.

현재 사용자(다수가 B2B 회사 고객)의 누적 friction 사례:

- 메모리 인증 → 앱 재시작마다 재로그인 (매일 사용자에게 매일 반복되는 friction)
- LAN-only stageview → 원격 멤버·외부 클라이언트 시연 불가
- 단일 PC 의존 → 회사 멤버 간 자료·하드웨어 공유 불가, 분산 시연·협업 어려움
- 단일 사용자 가정 → 회사 차원 결제·중앙 거버넌스 부재
- Pie 공유 → 파일 직접 전달·별도 채널, 회사 워크플로우와 분리

이 friction이 사용 빈도·만족도·충성도를 깎고 있음. 해소 시 사용자는 두 가지 가치를 얻고, 결과적으로 충성도·사용률 제고로 이어진다.

보조 컨텍스트 (시장 흐름): 프로토타이핑 디자인 도구 시장은 SaaS·실시간 협업으로 이동(Figma·Framer·Axure Cloud). ProtoPie Cloud도 SaaS. 우리 사용자도 같은 기대를 가지지만, 결정 동기는 우리 사용자의 friction 자체이지 시장 추종이 아니다.

클라우드화의 핵심 가치 — 사용자 관점 2가지

이 2가지가 클라우드화의 실제 이유. 특정 시나리오(분산 하드웨어·stageview 개선·Stage 등)는 이 2가지 위에서 가능해지는 결과물이지, 클라우드화의 이유 자체가 아님.

- 클라우드화를 통한 손쉬운 접근 — Connect 기능을 어디서나·어떤 디바이스에서나 접근. LAN bound·단일 PC bound·메모리 상태 손실·LAN 경유 시청 등 누적 friction 일괄 해소
- 빠른 파일 공유와 협업 — Pie를 팀원에게 즉시 공유·함께 작업 가능. 외부 클라이언트는 브라우저로 즉시 시청. 팀 단위 워크플로우의 자연스러운 토대

위 2가지 위에서 다양한 시나리오가 가능해짐 (S10 분산 하드웨어 / S11 stageview 개선 / S12 Stage 협업 등) — 이들은 결과 시나리오. SaaS 매출 모델. 플러그인 생태계 기반 등은 비즈니스 또는 운영 측 이득이지 사용자 관점 핵심 가치는 아님.

시장·경쟁 매트릭스, 24개월 시나리오, 시장 규모 추정, 통신 방식 대안 비교(P2P·VPN·LAN·SaaS Relay) 등 상세 전략 콘텐츠는  STRATEGY - WIP 참고. 본 PRD는 무엇을 만드는지에 집중.

1-4. 핵심 차이 매트릭스

§1-2 한계가 신규 앱에서 어떻게 해소되는지 정리. 신규 Connect는 레거시 모든 기능을 동등 이상으로 보장 (UX·stageview는 부수적으로 일부 개선되나 핵심 가치 아님). 앱은 새로 작성 (코드 마이그레이션 아님).

레거시 사용자 전환 메시지: “신규 클라우드 버전 Beta 시작, 레거시는 그대로 유지 — 강제 이전 없음”.

영역	Legacy	New Connect Cloud (Beta)
stageview 미러링	LAN 한정·모든 Pie 강제 시청·동기화 약함	선택적 Pie 시청 + Cloud 경유 분산 시청 + 자동 재연결
하드웨어 통합	단일 PC 한정	분산 하드웨어 협업 (각 PC의 하드웨어가 한 세션 통합)
로컬 서버 모드	Server 모드로 LAN 운영	로컬 모드 유지 + 클라우드 통신 모드 hybrid
외부 클라이언트 시연	화면 공유·VPN 의존	PIN 기반 Viewer 게스트 (브라우저로 즉시 접속)
플러그인 저작	외부 빌드 후 업로드	Bridge 내장 IDE
플러그인 공유	단일 PC 한정	Team 라이브러리 + Stage 단위 인스턴스
사용자/조직 모델	단일 사용자	Team → Device + Stage 격리 (Team 이 결제·격리·소속 root. User identity-membership 은 ProtoPie Cloud API SoT — Connect 미저장. JWT 에는 Cloud user ID 만, 권한 판단은 매 요청 Connect 서버 미들웨어가 Cloud API 호출로 처리)
인증 상태	메모리	영속 저장소 (DB·Redis) — 단 user 정체성·role 자체는 ProtoPie Cloud SoT
인증 모델	단일 (.lic 파일 + machine ID)	두 갈래 — (a) Cloud 모드 ProtoPie Cloud OAuth + Device JWT (@ppc/local-server schema) / (b) Local 모드 라이선스 키 (LicenseKey schema, 클라우드 연결 자체 차단)
Stage 모델	Group only	Cloud Stage (Team-shared, 멤버십·per-Stage role) + Local Stage (개인, 라이선스 모드). 로컬 앱 + Cloud 로그인 시 생성 시점에 선택 (변환 불가)
클라우드 연동	단방향 다운로드	양방향 동기화, 팀 자료 공유·협업
앱 코드베이스	모놀리식	모노레포 + 모듈 분리, 신규 작성
확장성	단일 프로세스	무상태 + 수평 확장
감사 로그	없음	모든 변경 작업 기록 (90일 보존)

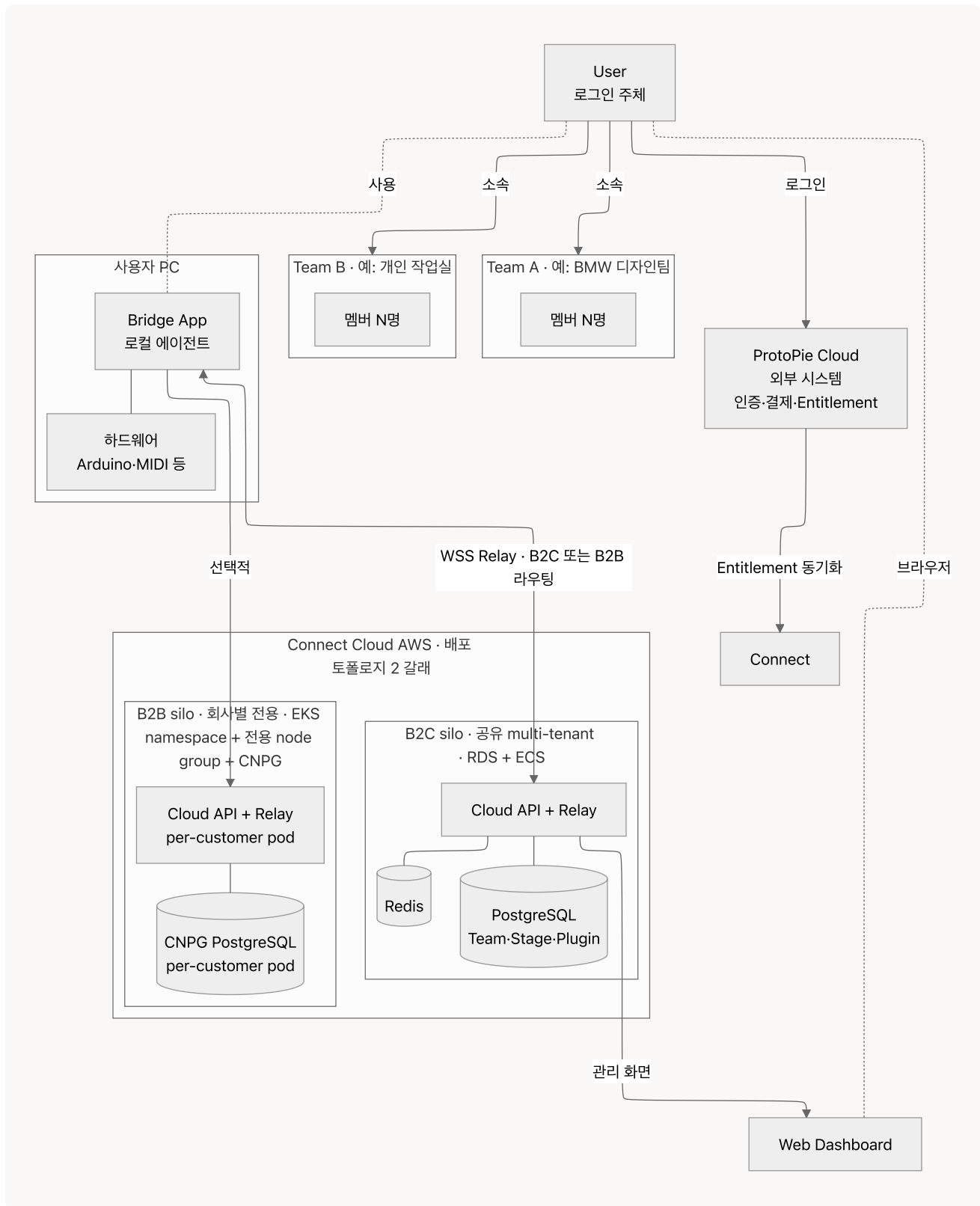
1-5. 레거시 컨셉·패턴 처리 (유지·폐기)

사용자 가시 기능 차이는 §1-4 매트릭스 참고. 코드는 새로 작성하되 개념·구조 차원에서 어떤 것을 이어받고 어떤 것을 버릴지 정리.

카테고리	처리
개념 유지 (구조 그대로 이어받을)	24시간 Auth Token 모델 · Manager 패턴 (DI) · ORM 사용 · PIN 기반 원격 접근
패턴 폐기 (구조 변경)	메모리 상태 저장 → 영속 저장소 / 데스크톱=서버 → Bridge·Cloud 분리 (로컬 서버는 Bridge 안에 유지) / 단일 사용자 → 멀티테넌트 / DB 강제 초기화 → 마이그레이션 도구 / 하드코딩 비밀 키 → 비밀 관리 / CORS 와일드카드 → 화이트리스트 / IP 판별 보안 → 토큰 인증 / 레거시 코드 재사용 → 신규 작성 (개념·UX만 이어받을)

1-6. 전체 시스템 한눈에 보기

6개 액터(User / Team / Device / Bridge / Cloud / ProtoPie Cloud)의 관계도. 본문에 나오는 모든 용어가 어디 위치하는지 빠른 파악용.
(v0.7.0: Tenant 표 제거, Team 이 root entity)



silo 두 갈래: 동일 schema·동일 코드, 호스팅/ 위치/만 다름. B2C 는 공유 multi-tenant (Team 별 격리 = RLS· `team_id`), B2B 는 회사별 전용 EKS namespace + per-customer node group + CNPG PostgreSQL pod (회사 단위 물리 격리).

2. 목표 (Goals)

2-0. Beta Scope — 6월 중순 개발 완료, 6월 말 출시

Beta는 Tech Preview 성격. 2026-06-15 개발 완료 + QA (6/16 ~ 6/30) → 2026-06-30 출시.

Beta 일정 — 개발 5주 (5/11~6/15) + QA 2주 (6/16~6/30)

단계	기간	비고
개발	5/11 ~ 6/15 (5주)	user research · 기획 · 디자인 · 엔지니어링 모두 포함하는 통합 실행 기간. 코드 작성만의 시간이 아님
QA	6/16 ~ 6/30 (2주)	통합 테스트·격리 검증·운영 회복성
출시	2026-06-30	

주차별 작업 내용은 본 PRD에서 사전 결정하지 않는다. 개발 kickoff + 데일리 미팅 + 위클리 미팅을 통해 단계적으로 결정. 본 PRD는 시작·종료일과 출시일만 고정하고, 그 사이의 작업 분배·우선순위 조정은 운영 회의체에 맡긴다.

2-1. 제품 목표

본 6개 목표는 \$0 Beta 최우선 2가지 (① 기존 Connect → 신규 앱 전환, ② 클라우드 환경에서 Connect 기능 사용 가능)의 하위 구체화. 충돌 시 \$0 priority가 tie-breaker. 아래 list 내 ★ 표시는 \$0 최우선 직결 항목.

- ★ 레거시 기능 동등 이상 보장 — Beta priority #1: 레거시 Connect의 모든 기능을 신규 앱에서 동등 이상의 품질로 제공. 코드는 재작성, 기능은 보존
- ★ 클라우드 SaaS 토대 + 로컬 모드 hybrid — Beta priority #2: 클라우드 통신 모드(분산 협업)를 기본으로 하되, 로컬 서버 단독 모드(오프라인·LAN)도 동시 지원
- 팀 협업 (priority #2 하위 구체화): 같은 팀 내 프로토타입·플러그인·세션 공유
- 플러그인 저작 환경 (priority #1 — 레거시 보존): Bridge 내 IDE에서 사용자가 직접 플러그인 작성
- UX-stageview 개선 (보조): 레거시 대비 온보딩·선택적 Pie 시청·반응성·에러 복구 등 사용자 경험 측면의 명시적 개선 (화질은 현재 수준 유지)
- @ppc/local-server 신규 작성 (priority #1·#2 공통 토대): cloud-poc의 Express 코드는 functional spec 참고용으로만 사용. Fastify + Socket.IO + Prisma 7 기반으로 처음부터 작성하고, 단일 라이브러리(@ppc/local-server)를 Desktop child fork 와 Cloud 컨테이너에서 동일하게 boot

2-2. 기술 목표

- 모놀리식 데스크톱 앱 → Bridge(로컬) + Cloud(중앙) 분리
- 메모리 상태 → 공유 저장소(Redis) 기반 영속 상태
- 단일 인스턴스 → 수평 확장 가능한 무상태(stateless) 서비스
- 단일 사용자 → Team → Device 2계층 정체성 모델
- 플러그인 무제한 권한 → 권한 선언 + 격리 환경 실행

본 PRD의 “확장성” = 기능 확장성(플러그인·모듈 추가) + 운영 확장성(트래픽 증가 시 인스턴스 추가, 수평 확장) 양쪽.

2-3. 정량 목표 (Beta)

Beta는 외부 SLA 약속 없음 — 최선의 노력. 측정 카테고리리는 정의하되, 임계값은 미도입. 레거시 Connect 사용 데이터 추출 후 baseline을 도출하고, 그 baseline 대비 비교 기준을 본 섹션에 추가한다.

결정 모델

- 레거시 데이터 추출 — 현재 운영 중인 ProtoPie Connect 사용 데이터(DAU·세션 수·기능별 사용 빈도·재방문률·latency 등) 수집·분석

- 2. **baseline** 도출 — 레거시 사용 패턴의 정량 기준선 정리
- 3. 비교 기준 정의 — Beta 신규는 baseline 대비 (a) parity / (b) +N% improvement 중 어느 방향을 목표로 할지 결정
- 4. 본 §2-3에 추가 — Go/No-Go 게이트 + 시나리오별·운영 임계값 명시

측정 카테고리 (임계값은 baseline 분석 후)

영역	측정 항목
Go/No-Go	S10 동시 활성 Device ≥2 세션 누적 카운트
S1 / S1-2 온보딩	가입자 Pie 생성률·첫 N일 retention
S10 분산 하드웨어	동시 활성 Device 세션 수·세션당 Device 수
S11 stageview	세션 누적·동시 시청자·latency p50/p95·재연결 성공률·선택적 Pie 시청 사용 비율
S12 Stage	Stage 생성률·Shared Stage 비율·초대→수락→첫 진입 시간·데이터 누출 0건 (정성)
운영 (API)	API p95 응답 지연
운영 (Relay)	Relay 메시지 지연 p95

운영 제약 (Beta scope)

항목	값
가용성 (SLA)	약속 없음. 단 AZ 단일 outage 시 RDS Multi-AZ failover로 분 단위 회복 (NDA에 명시)
동시 WebSocket 연결 (단일 클러스터)	≤ 100명 (Beta scope 제약)
멀티 리전	1개 (us-west-2)

| p95 = 100건의 요청 중 빠른 95건의 최댓값. “가장 빠른 95%가 이 시간 안에 처리됨”의 의미.

3. Non-goals (이번 범위 아님)

항목	이유
AI 기능 (Bridge·Cloud 양쪽)	별도 정책 결정 필요. 이번 Beta 제외
플러그인 마켓플레이스 / 공용 레지스트리	Team 별 프라이빗 공유만
3rd-party 플러그인 개발자 생태계	사용자 직접 작성만 허용
플러그인 결제·수익 분배	마켓플레이스 부재로 불필요
플러그인 import — URL/git 방식	파일 업로드만
플러그인 코드 서명 / 검수 프로세스	사용자 본인 책임 모델
On-prem / air-gapped 배포	Beta 는 안 함 (Connect-managed 만). 단 §6-5 의 B2B silo (EKS pod self-managed PG) 구조는 <i>Post-Beta on-prem</i> 진입 비용을 0 에 가깝게 만들기 위해 Beta 부터 채택 — 동일 Helm chart 를 customer 자체 OpenShift/Rancher 에 그대로 deploy 가능. STRATEGY 부록 A-3·A-11 evidence (BMW China·Huawei·BYD·GM·현대차 등) 의 향후 진입 경로 보존
Team 간 자원 공유 / 워크스페이스 초대	격리 우선
엔터프라이즈 운영 기능 일부 (SSO·SAML·SCIM·CMEK)	Post-Beta. 단 배포 토폴로지 차원의 B2B EKS silo 는 §6-5 와 ARCH §8-1 D-silo ADR 에 따라 Beta scope 에 포함 (KICKOFF §3-5 와 정합)
Local DB ↔ Cloud DB 동기화	Desktop local 모드 (🖥️ DB_SCHEMA_LOCAL - Draft) 와 Cloud silo (☁️ DB_SCHEMA - Draft) 는 schema-운영-데이터 완전 분리. Beta 안에서 Cloud↔Local 동기화·import-export 모두 미지원. 사용자가 라이선스 키 모드와 OAuth 모드 사이를 전환하면 새 환경으로 인지 (이전 작업물 자동 이전 안 함) — Post-Beta 검토

4. 타겟 사용자 / 페르소나

본 페르소나는 마케팅 분류가 아닌 제품 결정의 기준으로 활용. 각 페르소나의 핵심 욕구·페인 포인트·성공 기준을 명시.

Beta 타겟 페르소나

페르소나	비고
P1 개인 사용자 (B2C)	단독 사용 시나리오
P2 팀 멤버 (소규모 B2C 팀)	분산 하드웨어 협업(S10)·Stage 협업(S12) 시나리오 검증의 주체
P4 하드웨어 통합 개발자	P1·P2·P5에 분포하는 추가 역량. 플러그인 저작 검증
P5 B2B 팀 멤버 (회사·조직)	현재 Connect 사용자 base의 다수. 회사 단위 결제·멤버 간 자료 공유·중앙 거버넌스 검증

P3 결번: 이전 framing 에서 P3 = 엔터프라이즈 IT 관리자로 잡았으나, 엔터프라이즈 운영 기능 (SSO·SCIM·EKS·전용 인프라) 이 Post-Beta 로 미뤄지면서 (§3 Non-goals) Beta 타겟에서 빠짐. 번호 재정렬 대신 P3 결번으로 유지하여 외부 doc 의 P4·P5 cross-ref 안정성 보존. Post-Beta 진입 시 P3 재활성화 검토.

중요: 팀 페르소나(P2 소규모 B2C 팀 + P5 B2B 팀)가 Beta 타겟에 반드시 포함되어야 한다. §1-3 핵심 가치 #2 (빠른 파이 공유와 협업) 검증은 2명 이상 팀이 필요하므로, P1만으로는 협업·공유 시나리오(S1-2·S10·S12)를 입증할 수 없다.

B2B 비중: 현재 Connect 사용자 base의 다수가 B2B (회사·조직). Beta는 이들을 leave behind하지 않고 신규 클라우드 워크플로우를 함께 검증한다. 단, 엔터프라이즈 운영 기능(SSO·SCIM·EKS·전용 인프라)은 Post-Beta — Beta는 현재 B2B 고객의 핵심 워크플로우 검증에 집중.

공통 Team 구성: 모든 페르소나가 동일한 모델 — Team 이 root entity (결제·격리·UI 노출 단위), Team 단위 Addon 구매, 한 사용자가 여러 Team 에 동시 소속 가능 (membership SoT = ProtoPie Cloud). 페르소나별 차이는 사용 패턴·용구·페인.

4-1. P1: 개인 사용자 (B2C)

- **프로파일:** 프리랜서·학생·개인 사용자 (디자인·UX 배경). ProtoPie 능숙, 프로그래밍 보조적. 주 2~5회 프로젝트 단위 사용
- **핵심 욕구:** 본인 PC에서 하드웨어 연결 prototype 시연 · 프로젝트별 Addon 분리 청구
- **페인 포인트:** 외부 서비스 연동(Webhook·IFTTT) 위한 별도 도구 운영 부담 · 클라이언트 시연 환경 차이로 작동 실패
- **성공 기준:** 첫 30분 안에 Arduino 버튼 → prototype 화면ia 전환 작동
- **구매 동기:** 개인 프로젝트·학습·포트폴리오·클라이언트 시연

4-2. P2: 팀 멤버 (소규모 B2C 팀 — 2~5명, 프리랜서 콜라보·스타트업·연구 그룹)

- **프로파일:** ProtoPie 일상 사용. 매일 사용. 플러그인 코드 작성은 일부만
- **핵심 욕구:** 각자 PC 하드웨어를 한 prototype에서 동시 사용 (S10) · 동료 플러그인을 그대로 사용 · 원격 팀원과 동시 조작·리뷰
- **페인 포인트:** 레거시 단일 PC 강제 (시연 시 하드웨어 한 PC에 모아야 함) · 동료 플러그인 직접 복사 부담 · 팀 단위 결제·라이선스 관리 분산
- **성공 기준:** 2명이 각자 PC 하드웨어로 같은 prototype 세션 작동 (S10 시나리오)
- **구매 동기:** 빠른 파이 공유·팀 협업 효율·분산 하드웨어 통합

4-3. P4: 하드웨어 통합 개발자 (P1·P2·P5의 추가 역량, < 10%)

섹션 번호 §4-3 과 페르소나 번호 P4 의 mismatch: 페르소나는 P3 결번 (위 P3 결번 note 참조), 섹션 번호는 sequential. 일부러 다름.

- **프로파일:** 디자이너 또는 개발자 출신, 시리얼·MIDI·하드웨어 프로토콜 경험. 별도 페르소나 아닌 추가 역량
- **핵심 욕구:** Bridge IDE에서 직접 통합 코드 작성, 팀 동료와 그대로 사용
- **페인 포인트:** 기존엔 외부 ZIP 빌드 후 업로드 필요
- **성공 기준:** Bridge 내장 IDE에서 코드→실행→디버그→저장 한 곳에서

4-4. P5: B2B 팀 멤버 — 현재 Connect 사용자 base 다수

현재 Connect 고객 다수가 B2B. Beta는 이들을 leave behind하지 않고 함께 검증. **Beta scope** 단서: 엔터프라이즈 운영 기능 (SSO·SCIM·EKS·전용 인프라·on-prem)은 Post-Beta — Beta는 현재 B2B 고객의 핵심 워크플로우 (멤버 간 자료 공유·중앙 결제·기본 거버넌스) 검증에 집중.

- **프로파일:** 회사·조직(소규모~중견) 소속 디자이너·개발자 (자동차 OEM·가전·로봇 등 하드웨어 결합 업무 다수). 매일 사용 (업무 도구), 회사 보안·권한 정책 준수
- **핵심 욕구:** 회사 멤버끼리 Pie·플러그인 안전·빠른 공유 (핵심 가치 #2) · 회사 단위 사용권·청구 통합 · 회사 거버넌스(격리·감사·CORS) 안에서 작동 · 각자 PC 하드웨어를 회사 세션에 통합 (P4 역량 결합 시)
- **페인 포인트:** 레거시 단일 PC·LAN bound → 회사 차원 자료 공유·원격 운영 어려움 · 라이선스 분산 결제 부담 · 메모리 인증·격리 본재로 보안 부서 검토 issue · 멤버 변동 시 자료·플러그인 인계 어려움
- **성공 기준:** 회사 Team 안에서 멤버 (a) 동시에·(b) 안전하게·(c) 중앙 관리 하에 사용 — Pie·플러그인 공유 + Stage 협업이 회사 워크플로우에 정착
- **구매 동기:** 회사 단위 결제·운영 통합·보안 정책 준수·팀 자료 공유·협업 효율
- **Beta** 검증 우선순위: §1-3 핵심 가치 2가지 (P5에도 동일 적용)

5. 사용자 시나리오 (User Stories)

⚠ 본 §5은 user research·design 진행 전에 작성된 speculative pre-research baseline.

User research·기획·디자인은 5/11~6/15 개발 5주 통합 실행 기간 내 진행 — detailed flow·success criteria·error catalog는 본 research 출력으로 정제·교체될 것. 본 §5의 현재 detail은 작업 ground truth가 아니라, dev kickoff 시점 working hypothesis. 5주 기간 동안 §5 본문 이 user research·design 산출물로 evolve.

현재 PRD에 confirmed: ① 시나리오 ID·일람 (§6 기능 요구사항·§0 priority frame이 참조하는 cross-ref용) / ② 시나리오의 목적·검증 대상 (Beta priority·핵심 가치 mapping) / ③ user story 수준의 high-level “사용자가 무엇을 하고 싶은가” / ④ S-ERR·Designer 체크리스트는 운영·디자인 working list (research·design 진행 중 변동).

시나리오 일람

ID	시나리오	비고
S10	분산 하드웨어 협업	Priority #2 (클라우드 환경 작동) 검증 시나리오 중 하나
S1	개인 사용자 <u>온보딩</u> (B2C)	
S1-2	B2C 멤버 초대	소규모 팀 위주
S3	원격 시연 + Viewer 게스트	빠른 파이 공유·외부 클라이언트 시연 시나리오
S8	디바이스 분실 (최소 <u>revoke</u>)	
S11	stageview 미러링	레거시 동등 + UX 개선
S12	Stage 생성·초대·작업 격리	Discord 채널 모델

S1. 개인 사용자 온보딩 (B2C)

User story: 개인 사용자가 ProtoPie Cloud 계정으로 자신의 팀에 Connect Addon을 구매하고, Bridge를 설치·로그인해 첫 Pie·플러그인 작성에 도달한다.

관련 기능: §6-1 정체성·인증·권한 (Team 자동 생성 · Bridge Device 등록 · Entitlement 검증).

S1-2. B2C 멤버 초대

User story: 팀 소유자가 다른 멤버를 초대해 같은 Team에서 Connect를 함께 사용할 수 있게 한다.

관련 기능: §6-1 정체성·인증·권한 (Team N:M · 역할 기반 권한).

S3. 원격 시연 + Viewer 게스트

User story: 사용자가 Connect 미설치 상태의 외부 클라이언트(원격 PC·모바일)와 prototype을 원격으로 시연·인터랙션 한다.

관련 기능: §6-6 Relay (PIN 인증) · §6-7 Stageview Viewer. 핵심 가치 #2 (빠른 파이 공유)의 외부 게스트 측면.

Cite: Zillow Group (외부 공유), Disney Streaming (원격 UT).

S8. 디바이스 분실 / 변경

User story: 사용자가 특정 디바이스(노트북) 분실 시 그 디바이스만 차단하고, 새 디바이스에서 본인 작업을 이어간다.

관련 기능: §6-1 정체성·인증·권한 (Device-level 토큰 revoke).

S10. 분산 하드웨어 협업 (Priority #2 검증 시나리오 중 하나)

User story: 회사 멤버 둘이 각자 PC에 다른 하드웨어를 연결한 채로 같은 prototype 세션에 참여, 양쪽 하드웨어 입력이 한 prototype에 통합된다.

위치: S10은 Beta 목적이 아니라 Priority #2 (클라우드 환경에서 Connect 기능 작동)의 검증 신호 중 하나. 클라우드화의 목적은 “Connect 기능이 클라우드에서 작동” 자체이며 S10은 그 작동성을 보여주는 여러 시나리오 중 하나일 뿐.

관련 기능: §6-6 Relay (분산 하드웨어 이벤트 통합 dispatch).

Cite: BMW China·Mindray Medical·Inovance Technology·Scania (자동차/intelligent hardware/medical OEM의 분산 하드웨어 + 협업 needs).

S11. stageview 미러링 (레거시 동등 + 단순 개선)

User story: 사용자가 prototype 화면을 모바일·태블릿·웹 등 여러 시청 디바이스로 미러링하고, 시청자는 시청할 Pie를 선택할 수 있다.

위치: 레거시 사용자 가시 기능 중 하나. Beta에서의 stageview 개선은 단순 개선이며 핵심 가치 아님 (§0 priority frame).

관련 기능: §6-7 Stageview/Viewer + §6-6 Relay (stageview 미러링 채널 + 측정).

Cite: Toyota Japan·TomTom·Amazon (모바일·웹 임베드 needs), Canny 200+ vote (가장 오래·지지 많은 요청).

S12. Stage 생성·초대·작업 격리 (Discord 채널 모델)

User story: 사용자가 Team 안에 영구 작업 공간(Stage)을 만들고, 멤버 일부만 초대해 Pie·플러그인·Relay 세션을 그 Stage 단위로 격리한다.

관련 기능: §6-2 Stage (데이터 모델 · 멤버십 · 자원 격리 · 컨텍스트 자동 필터).

Cite: 핵심 가치 #2 (빠른 파이 공유와 협업)의 메인 컨테이너. 회사 워크플로우 정착 검증 (P5 페르소나 §4-4).

S-ERR. 에러·장애 카탈로그 (WIP placeholder)

⚠ 상세 에러 catalog는 user research·design 출력으로 정제·확장. 현재 PRD에 잠정 카테고리만 명시 — 자세한 에러 유형 정의·카피·복구 흐름은 5/11~6/15 dev 기간 내 디자이너·UX writer·엔지니어가 함께 정의.

기본 에러 카테고리 (placeholder): 인증 (E-AUTH-) · Entitlement·결제 (E-ENT-·E-PAY-) · 네트워크 (E-NET-) · 디바이스·USB (E-DEV-·E-USB-) · 플러그인 (E-PLG-) · Team·Relay (E-TEAM-·E-RELAY-) · OS 신뢰 체인 (E-OS-)

모든 에러는 운영 메트릭(오류·성능)에 기록. 빈도·영향 분석은 Beta 평가 시 활용.

Designer UX 보완 포인트 (WIP placeholder)

⚠ Designer 작업 결과로 채워질 영역. 5/11~6/15 dev 기간 내 user research 결과를 바탕으로 UX 흐름·카피·온보딩 가이드 등 디자이너가 정의. 본 PRD는 작업 단위·산출물 list를 담지 않음 — Figma·design tracking 도구에서 별도 진행.

5-A. 옛 detailed 시나리오 (참고 — research 출력 전까지의 working hypothesis)

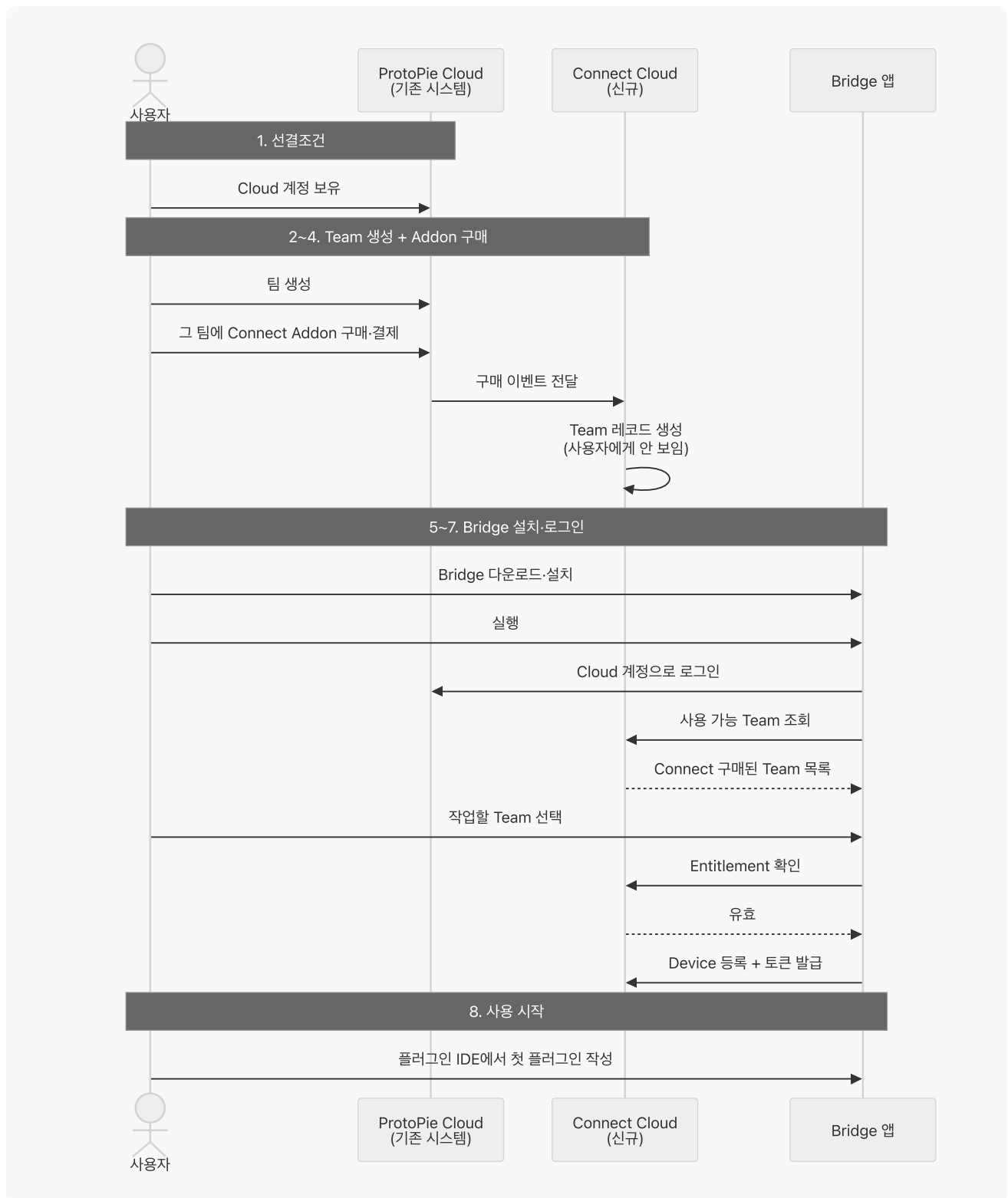
⚠ 본 §5-A 는 §5 시나리오 일람의 이전 detailed 버전으로, dev 5주 (5/11~6/15) 진행 중 §5 본문이 user research·design 산출물로 evolve 하면 자연 폐기됨. 현 시점 source of truth 는 §5 시나리오 일람 + 각 시나리오의 목적·검증 대상만. 본 §5-A 의 sequence-step-UX detail 은 user research 결과로 정제·교체 예정 — 구현 ground truth 아님.

시나리오 일람

ID	시나리오	비고
S10	분산 하드웨어 협업	Beta 핵심 가치 (클라우드화 검증)
S1	개인 사용자 온보딩 (B2C)	
S1-2	B2C 멤버 초대	소규모 팀 위주
S3	원격 시연 + Viewer 게스트	분산 하드웨어 협업 보조 축
S8	디바이스 분실 (최소 <u>revoke</u>)	
S11	stageview 미러링	레거시 동등 + UX 개선
S12	Stage 생성·초대·작업 격리	Discord 채널 모델

S1. 개인 사용자 온보딩 (B2C)

Connect는 **Team** 단위로 **Addon** 판매. 사용자가 자신의 팀을 만들고 그 팀에 Connect **Addon** 붙이는 흐름.



Addon 미구매 팀만 가진 사용자는 Bridge 로그인 시 “Connect 가능한 팀이 없음” 안내 + Addon 구매 유도 화면.

S1-2. B2C 멤버 초대 시나리오

1. 팀 소유자 Alice가 Connect Addon 구매한 Team A 보유
2. Alice가 Bob을 Team A에 초대 (ProtoPie Cloud의 팀 초대 기능)

3. Bob이 초대 수락 → Bob이 Team A의 멤버가 됨
4. Bob의 Bridge에 Team A가 사용 가능한 Team 으로 표시됨
5. Bob도 Team A 컨텍스트에서 Connect 사용 가능 (Alice 결제로 커버됨)
6. Bob의 권한은 Alice가 부여한 역할(Member/Viewer 등)에 따라 제한

S3. 원격 시연 (외부 클라이언트 게스트)

클라이언트 PC에 Connect 미설치 상태로 디자이너의 프로토타입을 원격에서 직접 인터랙션할 수 있는 시나리오. 레거시는 화면 공유.VPN으로만 가능했음.

1. 사용자가 자기 Bridge에서 Relay 방 생성
2. 외부 클라이언트(원격 PC, 모바일)에 방 코드 공유
3. 클라이언트가 PIN 인증 후 입장 (Team 멤버 아니어도 PIN 1회성 게스트 접속)
4. 디자이너의 프로토타입을 원격 클라이언트에서 실시간 인터랙션 (읽기.인터랙션만 가능, 업로드.설정 변경 불가)
5. 방 종료 시 게스트 세션 자동 만료

Beta 검증 지표: Relay 시연 성공률 / 외부 게스트 평균 세션 수 (임계값은 \$2-3 — 레거시 baseline 비교 후 정의).

S8. 디바이스 분실 / 변경

1. 사용자가 노트북 분실 신고 (Web Dashboard에서 디바이스 목록 → “이 디바이스 차단”)
2. Connect는 해당 Device의 토큰 즉시 revoke
3. 분실된 노트북에서 Bridge 실행 시 → 인증 실패, 차단 화면
4. 사용자는 새 노트북에서 동일 계정으로 로그인 → 신규 Device 등록
5. 기존 노트북에서 작성 중이던 플러그인은 마지막 자동 저장 시점까지만 복구 가능 (Cloud 동기화 정책 따름)

S10. 분산 하드웨어 협업 (Beta 핵심 가치 — 클라우드화 검증)

Beta의 본질적 검증 시나리오. 레거시는 한 PC에 모든 하드웨어를 모아 연결해야 했지만, 신규 Connect는 여러 PC에 분산된 하드웨어가 클라우드를 통해 하나의 프로토타입 세션에 동시 연결된다.

시나리오 예시: 2명 팀이 자동차 대시보드 프로토타입을 함께 시연

1. 김디자이너가 자기 PC(Bridge A)에 Arduino 연결 — 회전 다이얼 컨트롤러
2. 박개발자가 자기 PC(Bridge B)에 MIDI 컨트롤러 연결 — 페이더 입력
3. 둘 다 같은 Team 소속, 각자 Bridge에 로그인
4. 한 명이 같은 프로토타입 세션(Relay 방) 생성 후 다른 사람 초대
5. 김의 Arduino가 회전 → Cloud Relay → 박의 화면에서도 즉시 반영, 동시에 박의 MIDI 페이더가 김의 화면에도 반영
6. 양쪽 하드웨어 입력이 하나의 프로토타입 상태에 통합 → 클라이언트 시연
7. 세션 종료 시 각 Bridge는 자기 하드웨어만 정리

왜 이게 핵심인가:

- 레거시: 디자이너가 시연 전에 모든 하드웨어를 자기 PC로 모아 와야 했음 (협업 불가)
- 신규: 각자 자기 PC에 자기 하드웨어를 두고 클라우드로 연결. 원격 협업이 진짜 가능해짐
- Beta가 이걸 검증하지 못하면 Connect 클라우드화의 의미 자체가 없음

검증 방법 (운영 메트릭):

- 한 Team 안에서 동시 활성 Device 수가 ≥ 2 인 세션 카운트
- 동일 Relay 방에서 여러 Device의 하드웨어 이벤트가 동시 발생한 세션 카운트
- 분산 하드웨어 세션 평균 지속 시간.끊김률

열린 질문:

- 하드웨어 충돌 처리 (둘이 동시에 같은 입력 채널을 쓸 때)
- Latency 허용 범위 (한 Bridge → Cloud → 다른 Bridge — 목표값은 레거시 baseline 비교 후 정의)

- 누가 “마스터”인가 (방 호스트 Device 우선?)

시장 규모 추정 + 클라우드 vs 다른 통신 대안(P2P-VPN-LAN 브리징) 비교는 STRATEGY - WIP 참고.

S11. stageview 미러링 (레거시 기능 — UX 개선 포함)

stageview: 디자이너가 PC에서 실행 중인 프로토타입 화면을 모바일·태블릿·웹 등 여러 시청 디바이스로 동시에 미러링하는 기능. 레거시 Connect의 핵심 기능 중 하나로, 시연·리뷰·사용성 테스트에 광범위하게 사용됨.

Beta는 이 기능을 레거시 동등 이상으로 보장하면서 반응성·다중 디바이스 동기화 측면을 명시적으로 개선한다.

시나리오 흐름

1. 디자이너가 Bridge에서 프로토타입(.pie)을 로드해 실행 (Cloud 다운로드 또는 파일 업로드)
2. Bridge UI에서 “stageview 공유” 클릭 → QR 코드·짧은 URL 생성
3. 클라이언트가 자기 모바일·태블릿·노트북 브라우저로 URL 접속 (Connect 미설치)
4. 여러 디바이스가 동시에 같은 프로토타입 화면을 시청 (스테이지 변경·인터랙션·애니메이션 모두 동기화)
5. 디자이너가 프로토타입 인터랙션 → 모든 시청 디바이스에 즉시 반영
6. 리뷰 미팅 종료 시 디자이너가 세션 종료 → 모든 시청자 연결 해제

레거시 vs Beta 개선점

측면	레거시 Connect	Beta 신규 Connect	측정 지표
시청 대상	모든 Pie 강제 시청 (선택 불가)	시청자가 Pie 목록에서 시청할 Pie 선택	세션당 평균 선택 Pie 수 / 전환 횟수
반응성 (레이턴시)	LAN 한정 / 원격 시 지연 큼	Cloud Relay 경우 (목표값은 레거시 baseline 비교 후 정의)	입력 → 시청 디바이스 표시까지 latency p50/p95
다중 디바이스 동기화	화면별 개별 처리·드리프트 발생	단일 상태 소스 + 모든 시청자 동시 dispatch	동기화 드리프트 (디바이스 간 화면 차이 ms)
연결 안정성	끊김 시 수동 재접속	자동 재연결 + 마지막 상태 복구	끊김 → 자동 복구율
온보딩	Connect 설치·설정 필요	URL·QR 한 번 → 즉시 시청 (브라우저 만)	시청자 첫 화면까지 시간
시청 인원	같은 LAN의 소수	Cloud Relay 통해 분산 다수	동시 시청자 수

화질은 레거시 수준 유지 (현재 충분). 적응형 화질은 Beta scope 외.

stageview 검증 가설

- 선택적 Pie 시청 기능이 시연·리뷰 환경의 노이즈를 줄이고 사용자가 의도한 것에 집중하게 하는가
- 레거시 stageview 사용자가 신규에서도 동등 이상의 만족도를 얻는가
- “Cloud 경우”가 추가됐음에도 레이턴시가 레거시(LAN)보다 크게 나빠지지 않는가
- 온보딩 마찰(설치 없이 URL만)이 시청자 진입 이탈률을 낮추는가
- 다중 디바이스 동기화가 실제 리뷰·시연 환경에서 가치 있는가

stageview Beta 성공 기준

정량 임계값은 미도입. 레거시 Connect 사용 데이터 추출 후 baseline 비교 기준으로 추가 (§2-3 참고).

측정 카테고리:

- stageview 세션 누적 카운트
- 세션당 동시 시청자 수
- 입력 → 시청 latency p50 / p95
- 자동 재연결 성공률

- 선택적 Pie 시청 기능 사용 비율 (선택 vs 전체 시청)
- 레거시 stageview 사용자 인터뷰 만족도 (정성)

열린 질문

- 시청자 인증 정책 (공개 URL vs Team 멤버 한정 vs 토큰 게스트)
- 모바일 OS 백그라운드 시 절전 모드 처리 (자동 일시정지·복귀)
- 시청자 측 인터랙션 허용 여부 (시청 전용 vs 양방향 — Beta는 시청 전용으로 한정)
- 선택적 Pie 시청 UX (목록 표시·전환 방식·다중 선택 가능 여부)

S12. Stage 생성·초대·작업 격리 (Discord 채널 모델)

Beta 검증 시나리오. 한 Team 안에서 사용자가 영구 작업 공간(Stage)을 만들고, 멤버 일부만 초대하고, Pie·플러그인·Relay 세션을 그 Stage 단위로 묶는다. 디스코드 서버 안의 채널과 같은 정신.

시나리오 흐름 (B2C 김디자이너)

1. 김디자이너 가입 → 자동으로 “내 작업실” Private Stage 1개 생성 (본인만 접근)
2. 새 Pie 만들기 → 기본은 “내 작업실”에 배치. Pie는 다른 사람에게 안 보임
3. 친구 박디자이너와 협업 시작 → “박과의 공동작업” Shared Stage 생성. Team 멤버로 박디자이너 초대 후 그 Stage 멤버로 추가, role=editor
4. 협업용 Pie를 그 Shared Stage 에 옮김 → 박디자이너에게만 보임 (“내 작업실”의 다른 Pie는 박에게 안 보임)
5. 그 Shared Stage 안에서 stageview 시작 → 박디자이너 디바이스로 미러링
6. 외주 클라이언트 시안 공유 → 별도 “외주 시안 공유” Shared Stage 생성, 클라이언트는 Team 게스트로 초대 (role=viewer). 김디자이너의 다른 Stage 들은 안 보임
7. 작업 종료 후 Stage 삭제 또는 archived 처리

Stage 검증 가설

- 사용자가 Stage 모델을 자기 멘탈 모델로 받아들이는가 (디스코드/노션 익숙함 vs 신규층)
- “Team 안인데도 Stage 마다 멤버·자원이 다르다”는 격리 모델이 혼란을 주지 않는가
- 가입 직후 “내 작업실” 자동 생성이 빈 IDE 진입의 첫 30분 UX 공백(첫 플러그인 진입)을 줄이는가
- Shared Stage 가 Beta 분산 하드웨어 협업과 stageview의 컨테이너로 자연스럽게 작동하는가

Stage Beta 성공 기준

정량 임계값은 미도입. 레거시 baseline 비교 후 §2-3에 추가.

측정 카테고리:

- 가입 사용자 중 첫 N일 안에 Pie를 자기 Stage 에 배치한 비율
- 활성 Team 중 Shared Stage 를 생성한 비율 (협업 의도 검증)
- Stage 멤버 초대 → 수락 → 첫 진입까지 소요 시간
- 데이터 누출 0건 (다른 Stage 자원이 의도치 않게 노출되는 사례) — 정성 0-tolerance

Stage 열린 질문

- 외부 게스트 모델: Team 비-멤버를 Stage 에 초대 가능한가? 토큰 게스트로 한정?
- Pie 이동·복사 정책: Beta는 Stage 간 이동 불가, 새 Pie 생성으로 한정?
- Relay 방·stageview 시작 컨텍스트: Stage 외부에서도 시작 가능한가? Beta는 Stage 컨텍스트 필수로

S-ERR. 에러·장애 카탈로그

위 시나리오들은 happy path. 실제 운영에서 발생할 수 있는 실패·장애 흐름과 사용자 안내 정책.

코드	상황	사용자에게 보이는 화면	시스템 동작	복구
E-AUTH-1	ProtoPie Cloud 로그인 실패	“로그인 실패. 다시 시도” modal	Bridge 시작 화면 유지	사용자 재시도
E-AUTH-2	Device 토큰 만료	“세션이 만료되었습니다. 다시 로그인”	자동 로그아웃 + 로그인 화면	재로그인
E-AUTH-3	Device 토큰 차단됨 (분실 신고 등)	“이 디바이스는 차단되었습니다. 관리자에게 문의”	Bridge 잠금	관리자가 차단 해제
E-ENT-1	Team Entitlement 만료·결제 실패	“Connect 사용 기간이 끝났습니다” 차단	즉시 읽기 전용 모드	결제 갱신 시 즉시 복구
E-ENT-2	ProtoPie Cloud 장애로 Entitlement 검증 불가	(무음)	캐시된 토큰으로 24h 읽기 전용 grace	ProtoPie Cloud 복구 시 자동
E-NET-1	Cloud 연결 끊김 (네트워크)	우측 하단 토스트 “Cloud 연결 끊김. 재 연결 시도 중...”	자동 재연결 (지수 백오프)	네트워크 복구
E-NET-2	Relay 방 연결 끊김	“방에서 끊어졌습니다. 재입장”	5초 후 자동 재입장 시도	자동 또는 수동
E-DEV-1	Device 등록 실패 (네트워크·시계 오차)	“디바이스 등록 실패: [이유]” + 재시도 버튼	등록 화면 유지	사용자 재시도
E-PLG-1	플러그인 실행 크래시	플러그인 카드에 빨간색 + “재시작”	자동 1회 재시작 후 실패 시 정지	사용자 재시작 또는 코드 수정
E-PLG-2	플러그인 manifest 잘못된 (Import 시)	“플러그인 형식 오류: [상세]”	Import 거부	사용자가 zip 수정 후 재업로드
E-USB-1	USB 디바이스 권한 없음 (macOS)	“Arduino 사용 권한 필요” + 시스템 설정 deep-link	디바이스 미인식 상태 유지	사용자가 권한 부여
E-PAY-1	결제 → Bridge 가시성 latency	“결제 처리 중. 1~2분 후 새로고침” + 새로고침 버튼	폴링 (15초 간격)	1~2분 내 자동 또는 수동 새로고침
E-TEAM-1	Connect 사용 가능한 Team 없음	“Connect Addon 구매하기” 안내 화면 + ProtoPie Cloud 결제 직접 이동 링크	Bridge 잠금 (목록 외 진입 불가)	결제 후 자동
E-RELAY-1	Relay 방 만료·종료됨	“이 방은 종료되었습니다” + 새 방 생성 버튼	방 목록으로 돌아감	새 방 생성
E-OS-1	macOS Gatekeeper / Windows SmartScreen 경고	OS 기본 경고 화면 (앱 검증 안 됨)	(Bridge가 직접 안내 못 함)	OS 신뢰 후 재실행 / 코드 서명 정상화

위 에러는 모두 운영 메트릭(오류·성능)에 기록. 빈도·영향 분석은 Beta 평가 시 활용.

Designer의 UX 보완 포인트 체크리스트

시나리오에서 도출된 첫 30분 UX 보완 포인트. Designer가 다듬어갈 영역.

보완 포인트	방향	협업
첫 플러그인 진입 (빈 IDE)	Starter 템플릿 (Arduino serial · MIDI 1종씩) + 5분 작동 가이드 + 온보딩 체크리스트	Designer + Bridge Eng
Pie 로드 흐름	Bridge에서 Pie(.pie) 파일 import / Cloud 다운로드 화면 mock	Designer + Bridge Eng
Team UI 카피	화면별 카피 가이드 ("Workspace" / "Team" 노출 정책. "Tenant" 용어는 내부 schema 잔재로도 절대 UI 노출 금지 — v0.7.0 부터 schema 에서도 제거 됨)	Designer + UX Writer
결제 → Bridge 가시성 동기화 UX	"결제 처리 중" 안내, 재시도 버튼, 예상 대기시간 카피, 1~2분 후 자동 새로고침	Designer
OS 신뢰 체인 흐름	macOS Gatekeeper / Windows SmartScreen 경고 후 사용자 가이드 (스크린샷·설명 포함)	Designer + Bridge Eng
실패 복구 UX	인증·네트워크·디바이스 에러 화면·카피·재시도 동선 (에러 카탈로그 참고)	Designer + UX Writer
"Connect 가능한 팀 없음" 화면	Addon 미구매 사용자 진입 시 화면, ProtoPie Cloud 결제로 직접 이동 링크	Designer + Marketing

추가로 다음을 영역:

- Team 전환 상태 머신을 모달·토스트·인라인 중 어떤 UI 패턴으로 표현할지 결정
- 에러 카탈로그의 모든 에러 상태에 대한 카피·아이콘 정의
- Web Dashboard 최소 기능 화면 (Team 전환·Device 관리·자기 활동 로그·결제 안내)

6. 기능 요구사항

⚠ 본 §6은 user research·design·엔지니어링 진행 전에 작성된 speculative requirements list.

구체 기능 요구사항(F-* IDs detail)은 5/11~6/15 개발 5주 통합 실행 기간 내 user research·기획·디자인·엔지니어링 결과로 정제·교체. 본 §6의 현재 detail은 작업 ground truth가 아니라, dev kickoff 시점 working hypothesis. 5주 기간 동안 §6 본문이 산출물로 evolve.

현재 PRD에 confirmed: ① 모듈 구조 (8개 sub-section) / ② 핵심 컨셉 (Team 이 root entity, v0.7.0 — 결제·격리·UI 노출 단위. Team·Device 2계층 · Discord 채널 모델 Stage · hybrid 로컬+클라우드 · Multi-tenancy 격리) / ③ 명시 확정 기능 (F-BRG-15-F-BRG-16-F-BRG-17 — 사용자 confirmed) / ④ 레거시 47개 기능 동등 보장 (P#1).

F-* ID prefix 체계 (traceability 기반)

각 §6-X 의 확정 컨셉 bullet 은 향후 F-* ID 가 부여되며, prefix 는 다음 규칙으로 통일:

§6-X	F-* prefix	영역
§6-1	F-IDM-*	정체성·인증·권한 (Identity Management)
§6-2	F-STG-*	Stage·Group·Pie (Stage hierarchy)
§6-3	F-PLG-*	플러그인 시스템
§6-4	F-BRG-*	Bridge 앱 (현재 F-BRG-15·16·17 만 확정)
§6-5	F-CLD-*	Cloud 서버·silو·배포
§6-6	F-REL-*	Relay 실시간 통신
§6-7	F-VWR-*	Stageview·Viewer
§6-8	F-AUD-*	감사·관찰성

번호 (* 부분) 는 dev 5주 진행 중 user research·design 산출물에 따라 부여·재정렬. Schema 의 cross-ref 는 F- ID 가 확정될 때까지* 섹션 ref (§6-1·§6-2 식) 로 임시 대체 (🔗 DB_SCHEMA - Draft Open Questions 의 PRD ref 컬럼 참조).

6-1. 정체성 / 인증 / 권한

확정 컨셉:

- Team 이 root entity (v0.7.0 — 결제·격리·UI 노출 단위. 이전 Tenant 표 흡수)
- Team → Device 2계층 (User identity·membership 은 ProtoPie Cloud API SoT, Connect 미저장)
- 사용자 N:M Team 소속 (Slack 워크스페이스 모델)
- 영속 인증 (메모리 → DB·Redis로 교체) — 서버 재시작해도 로그인 유지
- Device-level 토큰 (디바이스 분실 시 그것만 차단, 사용자 전체 X)
- PIN·24시간 토큰 인증 흐름 — 레거시 동등 (저장소만 영속화)

Team role (Owner/Admin/Member/Viewer) 정책 — v0.7.0 확정:

- Team role 의 source of truth = ProtoPie Cloud API. Connect schema 에는 Team role enum 도, TeamMembership 표도 두지 않는다. JWT 에는 Cloud user ID (sub) 만 들어 있고, 어느 Team 의 어떤 role 인지 는 매 요청 Connect 서버 미들웨어가 Cloud API 호출로 판별 (§1-4·§6-1 Identity 정책)
- 따라서 PRD 는 Team role 4 종 (Owner/Admin/Member/Viewer) 의 존재·매핑 시점 만 spec 하고, 권한 매트릭스의 detail (Owner 가 무엇을 할 수 있고 Viewer 가 무엇을 못하는지) 은 Cloud team-management API 의 SoT 를 따른다 — Connect 측에서 별도 정의·검증 안 함
- Stage role 은 Connect Stage 자체의 격리 단위이므로 Connect schema 가 owner·editor·viewer 3 종을 자체 정의 (§6-2)

WIP: Team 전환 UX·토큰 lifecycle·결제 변동 처리·로컬 전용 인증 detail 은 dev 기간 내 정의.

6-2. Stage (작업 공간 — Discord 채널 모델)

확정 컨셉:

- Team 안의 영구 작업 공간 (Private + Shared 2종)
- Pie·플러그인 인스턴스·Relay 세션·stageview 채널 격리 단위
- 가입 시 자동 Private “내 작업실” 생성
- 명시적 멤버십 (Team 멤버 ≠ Stage 멤버)
- UI 카피: “Stage”. “Workspace”. “작업 공간” 노출, “Tenant” 용어는 schema 에서 제거됨 — UI 노출 금지 (v0.7.0). Room 명칭은 v0.8.0 부터 폐기 — UI 노출 금지

계층 관계 (1:N):



- 한 **Team** 은 여러 **Stage** 를 가질 수 있다 (1:N)
- 한 **Stage** 는 여러 **Group** 을 가질 수 있다 (1:N, 선택)
- 한 **Stage** 는 여러 **Pie** 를 직접 가질 수 있다 (Stage root, 1:N)
- 한 **Group** 은 여러 **Pie** 를 가질 수 있다 (1:N)
- 한 **Pie** 는 **Stage root** 또는 **Group** 한 곳에만 위치 (**group_id** NULL 또는 NOT NULL, mutually exclusive)
- 역방향은 모두 N:1 — 한 **Pie** 는 한 **Stage** 에 속하고, 한 **Group** 도 한 **Stage** 에 속한다

Stage 생성 flow (Cloud Stage vs Local Stage):

Stage 는 어디에 저장되는지에 따라 두 종류로 분리되며, 생성 시점에 선택 하고 변환 불가.

- **Cloud Stage** — Cloud Postgres 의 Team-scoped row. Team 멤버 초대·협업·Cloud Pie 다운로드 가능. 로그인 + Cloud 계약 필요
- **Local Stage** — Desktop 앱 안 embedded Postgres 의 personal row. 단일 사용자·단일 머신. Cloud 연결 없이도 작동 (License-only 모드 호환)

로컬 앱 + Cloud 로그인 상태에서 사용자가 새 Stage 를 만들 때 *Cloud Stage / Local Stage* 둘 중 하나를 선택. License-only 모드 (Cloud 연결 차단) 는 Cloud Stage 옵션 자체가 비활성. Cloud↔Local 변환은 Beta scope 외 (§3 Non-goals).

Pie 배치 레이어 — Group (v0.9.0 신설)

Stage 안에서 Pie 들을 **폴더 단위로 묶는 선택 레이어**. Pie 는 두 위치 중 하나에 존재:

- **Stage root** 직접: Pie 가 Stage 바로 아래 (Group 없이도 사용 가능 — 기본 패턴)
- **Group** 안: Pie 가 특정 Group 폴더 안

제약:

- *1 level only* — Group 안에 또 다른 Group 못 들어감 (Stage → Group → Pie 까지만)
- *Cascade* — Group 삭제 시 안의 Pie 도 함께 삭제 (Notion 폴더 스타일)
- *선택 사용* — Stage 가 group 없이 root 만으로 운영 가능
- *Pie 위치 mutually exclusive* — Pie 는 Stage root 또는 Group 한 곳에만. 동시에 양쪽에 못 둬
- *정렬 (display_order)* — Stage root 와 각 Group 내부가 각각 독립 namespace. drag-and-drop 재정렬 시 동일 순서값이 root-Group 에 동시에 존재 가능 (UNIQUE 없음)

Pie 의 버전 추적 — PieVersion

같은 .pie 파일이 여러 번 업로드되거나 (Studio 재저장·Cloud 동기화), source 가 바뀌어도 (upload:studio:cloud_download:html_import) Pie row 자체는 1 개이고 **PieVersion** 이 매 업로드/import 마다 1 row 씩 누적 — (team_id, pie_id, version) UNIQUE, partial UNIQUE **pie_id WHERE is_current = true** 로 활성 버전 1개 강제 (DB_SCHEMA §3-11). 디버거 trace 와 외부 게스트 시청 세션도 모두 PieVersion 단위 로 추적 — 같은 Pie 가 사용자가 새 버전 업로드해도 진행 중인 세션 영향 없음.

Cloud Stage / Local Stage × Private / Shared 조합

Stage 의 두 분류 차원은 어디에 저장되는지 (Cloud vs Local) 와 멤버십 여부 (Private vs Shared) 로 독립:

차원	Cloud Stage	Local Stage
Private (단일 멤버)	✓ 가입 시 자동 생성되는 내 작업실, Cloud 저장, Team 멤버 본인만	✓ Desktop 앱 default Stage, single-user·single-machine. Shared 자체가 불가
Shared (다중 멤버)	✓ Team 멤버 명시 초대, 협업 핵심	× (Local 은 멤버십 자체 없음)

즉 *Local Stage = Private only*, *Cloud Stage = Private 또는 Shared*. License-only 모드 (Cloud 연결 차단)에서는 *Local Private Stage* 만 사용 가능.

Stage 부속 자산:

- **StageAsset** (DB_SCHEMA §3-8): Pie 가 참조하는 *Stage scope 외부 자산* — Unity 빌드·web_view html-camera feed metadata 등. 레거시 **SUnityLayer** · **SCameraLayer** 흡수. 자산 binary 는 S3, schema 는 metadata 만
- **TeamFont** (DB_SCHEMA §3-12): Team 전용 폰트 라이브러리. Pie 안에서 system font 외에 Team 이 업로드한 폰트 사용 가능. license 검증된 폰트만 (Beta scope 외 검증은 *사용자 본인 책임*)

Stage role (Connect 자체 정의 — Team role 과 별개):

Stage 는 Connect schema 의 격리 단위이므로 Stage role enum 을 Connect 가 직접 정의:

role	설명
owner	Stage 생성자 (Team OWNER 와 별개). Stage 삭제·멤버 관리 가능
editor	Pie·Group 편집·플러그인 인스턴스 조작 가능
viewer	읽기 전용

Team role (Cloud SoT) 과 Stage role (Connect SoT) 은 독립 **RBAC** — Team OWNER 가 자동으로 모든 Stage 의 owner 가 되지 않음. 한 사용자가 같은 Team 에서 Team MEMBER + Stage owner 조합도 가능.

WIP: 멤버십 모델·자원 격리 메커니즘:Stage 전환 UI·외부 게스트 모델:archived 정책·운영 메트릭은 dev 중 정의.

6-3. 플러그인 시스템

확정 컨셉:

- **Bridge** 내장 IDE에서 작성·실행 (Beta는 Bridge 전용)
- **Cloud (Web Dashboard)** 미노출 — Beta 에서 플러그인은 Bridge 안에서만 보고·실행. Web Dashboard 의 Cloud Stage 뷰에는 *플러그인 인스턴스 표시·실행 UI 없음*. 사유: Beta 의 플러그인 검수·서명 정책이 *사용자 본인 책임* 모델이라 (§3 Non-goals), 외부 노출 면적을 Bridge 로만 한정해 보안 표면 축소
- **Team** 라이브러리 + **Stage** 단위 인스턴스 모델
- **Manifest** 기반 권한 선언

WIP: 권한 강제 메커니즘·격리 환경·의존성 설치 정책·S3 storage 정책 detail은 dev 중 정의.

6-4. Bridge 앱

확정 컨셉:

- 로컬 하드웨어 연결 (USB·Serial·MQTT) — 레거시 동등 보장
- 로컬 서버 + 원격 **Cloud Relay** 양쪽 동시 지원 (*hybrid*)
- **빌드 보호** (코드 난독화·라이선스 우회 차단·변조 빌드 거부)
- **Web Dashboard** ↔ **Bridge** 로컬 인증 통신
- 자동 업데이트 (디지털 서명 검증)

확정 기능 (사용자 confirmed):

ID	요구사항
F-BRG-15	레거시 ProtoPie Studio 연동 동등 보 장: ① STU-1 (Studio SocketIO 연 결) — Studio의 IP:port:sessionToken을 입력으로 받 아 Server가 Studio로 SocketIO 연결 을 열고 양방향 메시지 라우팅. ② STU- 2 (Studio .pie 업로드) — Studio 가 Server로 .pie 파일을 multipart POST 업로드 → 저장 → 즉시 실행 가 능. 신규 동작(Studio 자동 발견.페어링 승인 UX 등)은 추가하지 않음 — 레거시 그대로
F-BRG-16	<u>Debugger</u> 모드 — 데이터 흐름·노드 <u>인터랙션</u> 시각화 (Beta 신규): <u>Bridge</u> <u>UI</u> 에서 실시간으로 (a) 하드웨어 이벤트 → 플러그인 → 프로토타입 메시지 흐 름, (b) Pie 안의 노드 기반 인터랙션 트 리거 시퀀스를 시각화. <u>정지·재생·step-</u> <u>through·이벤트</u> 타임라인 기록. <u>Cite:</u> <u>Scania·내부 피드백 (Connect</u> <u>Debug Log</u> 개선 필요 —  <u>STRATEGY - WIP</u> 부록 A-9)
F-BRG-17	<u>Socket</u> 기반 <u>HTML</u> 프로토타입 <u>import</u> (Beta 실험적): 외부 HTML 프 로토타입(WebSocket·Socket.IO 클 라이언트 코드 포함)을 Bridge에 import해 ProtoPie 메시지 프로토콜로 연결. <u>Beta scope</u> 단서: ① 보안 정책 (샌드박스·CORS·permission scope) 미확정 — Week 2 ADR (D- 항목). ② Beta 진행 중 우선순위 따라 deferral 가능. ③ 파일 업로드만 (URL/git import 거절은 §3 Non- goals 정함)

WIP: Cloud 서버 주소 입력·Plugin IDE detail·Team 전환 UX·자동 업데이트 정책 detail은 dev 중 정의.

6-5. Cloud 서버

확정 컨셉:

- 이중 silo 배포 모델 (ARCH §8-1 D-silo ADR):
 - B2C silo — AWS ECS Fargate (task ≥2 Multi-AZ) + RDS Multi-AZ (managed). 모든 B2C Team 이 단일 환경 공유 (애플리케이션 레
벨 격리)
 - B2B silo — customer 회사마다 전용 silo. EKS namespace 안 PostgreSQL pod (CloudNativePG operator, self-managed) + 앱
pod, 1 회사 silo (그 안에 1+ Team 공존 가능). 회사 식별은 배포 metadata (PPC_COMPANY_NAMESPACE env var) 가 표현 — schema 예
회사 entity 없음 (v0.7.0)
- Schema·코드 통일 — Prisma schema 한 벌, @ppc/local-server 한 벌, RLS 양쪽 모두 ON. 차이는 배포 토폴로지 + DB 운영 책임만 (B2C
는 AWS managed, B2B 는 self-managed)
- Fastify + Socket.IO + Prisma 7 + PostgreSQL + Redis 스택 (@ppc/local-server 라이브러리에 통합 — Desktop child fork·B2C 컨테
이너·B2B EKS pod 공통 boot)
- 무상태 서비스 + Redis 메시지 동기화 (다중 인스턴스)
- 영속 저장만 (메모리 저장 금지)
- Team 격리 자동 필터 (모든 테이블에 team_id 컬럼, 잘못된 Team 조회 차단). B2B silo에선 회사가 1곳이라 RLS 가 deep-defense 역할 (silo
안 다중 Team 분리에는 여전히 1차 방어)
- DB 마이그레이션 도구 (강제 초기화 금지). B2B silo N개에 대한 fan-out 메커니즘은 ARCH §8-1 D-FanOut ADR
- 비밀 키 외부 저장소 (AWS Secrets Manager / External Secrets Operator)

WIP: 자세한 API endpoint-DB 마이그레이션 fan-out 절차-silo routing registry 위치 (control plane DB vs ProtoPie Cloud 위임)-cross-silo identity 통합-역할 모드(전체/API/Relay 전용) 분기는 Week 1~2 ADR.

6-6. Relay (실시간 통신)

확정 컨셉:

- **Stage** 컨텍스트 안에서 방 생성·관리 (Team 격리 자동 상속). 영구 Stage vs ephemeral Relay 방 구분 — Stage 는 Schema **Stage** row (영구), Relay 방은 Schema **RelaySession** row (세션 생애만, **room_code** 단명 코드)
- **PIN 게스트 (외부 viewer)** = Schema **ViewerInvite** (PIN·공유 링크·QR 같은 token 의 표현) + **ViewerSession** (게스트 접속 세션). Team 외부 인원이 특정 Stageview Channel 만 보기 위한 1회·다회용 토큰. Schema §3-20·§3-21 참조. Bridge↔Device 페어링용 **Pin** entity (§3-4)와는 별도 도메인
- 다중 Cloud 인스턴스 메시지 동기화 (Redis pub/sub)
- 서버 종료 시 클라이언트 자동 재연결
- **Bridge** 로컬 모드도 동일 Relay 프로토콜 (Cloud 없이도 시연)
- 분산 하드웨어 이벤트 + stageview 미러링 모두 같은 dispatch 메커니즘

WIP: 메시지 envelope-idempotency-하드웨어 충돌 처리-지연 측정-다중 Bridge 동시 활성화 정책은 dev 기간 ADR (D2 + D7 + D12 — §0 Open Questions).

6-7. Stageview / Viewer

stageview는 레거시 사용자 가시 기능 중 하나. 신규 앱은 레거시 동등 기능 보장이 우선이며, 반응성·다중 디바이스 동기화는 부수적으로 일부 개선 (Beta 핵심 가치 아님).